ADM/ADP32F036 系列 BOOT/APP 程序核查与修改指南

Rev. 1.1，8/2026

技术文件修订记录

版本号	时间	描　述	制作人	审核人	批准人
V1.0	2026/8		吴武章	徐航	林钊漳
V1.1	2026/8		胡从春	徐航	林钊漳

目录

1 BOOT 程序 4
1.1 更改 Flash 等待时间 4
1.2 更改中断向量表 4
1.3 更改保留函数 5
1.4 确认软件配置 Flash 擦除时间及更新 FlashAPI 5
2 APP 程序 6
2.1 更改 Flash 等待时间 6
2.2 更改中断向量表 7
2.3 更改保留函数 7
2.4 休眠函数例程建议 8
2.5 确认软件配置 Flash 擦除时间及更新 FlashAPI 9
2.6 休眠/VM 下电到 5.3V 前，确保 SCL 为输入模式，SDA 为 TZ 模式 10

1 BOOT 程序

1.1 更改 Flash 等待时间

100M 主频，Flash 等待时间 3，3，5 改成 4，4，5。

```
//---------------------------------------------------------------------------
void InitFlash(void)
{
   EALLOW;

   //Enable Flash Pipeline mode to improve performance of code executed from Flash.
   //FlashRegs.FOPT.bit.ENPIPE = 1;
   FlashRegs.FOPT.all = 0x01;
   //Set the Random Waitstate for the Flash
   FlashRegs.FBANKWAIT.bit.RANDWAIT = READ_RAND_FLASH_WAITE;
   //Set the Paged Waitstate for the Flash
   FlashRegs.FBANKWAIT.bit.PAGEWAIT = READ_PAGE_FLASH_WAITE;
   //Set the Waitstate for the OTP
   FlashRegs.FOTPWAIT.bit.OTPWAIT = READ_OTP_WAITE;

   //Set number of cycles to transition from sleep to standby
   //FlashRegs.FSTDBYWAIT.bit.STDBYWAIT = 0x01FF;
   //Set number of cycles to transition from standby to active
   //FlashRegs.FACTIVEWAIT.bit.ACTIVEWAIT = 0x01FF;
   EDIS;

   asm(" RPT #7 || NOP");
}

/*************************************************************
    定义Flash 等待时间：035B/036 相同。
*************************************************************/
#define   READ_RAND_FLASH_WAITE      4       //Flash中运行的等待时间
#define   READ_PAGE_FLASH_WAITE      4       //
#define   READ_OTP_WAITE             5       //OTP中读数据的等待时间
```

1.2 更改中断向量表

地址 0xD00~0xD05 不属于中断向量，需跳过。

添加 pPieTable += 3，将 128 改成 125，需要确定 PieVectTable 的起始地址为 0xD00。

HardwareSetup.c

```
Parameters: None
Returns: None
*****************************************************************************/
void PieVectTable_Initialize(void)
{
    int16 i;
    PINT *pPieTable = (void *) &PieVectTable;  // Pointer to PIE vector table

    EALLOW;  // Enable writes to protected registers
    pPieTable += 3;
    // Initialize all 128 interrupt vectors to the default reserved ISR
    for(i = 0; i < 125; i++)
    {
        *pPieTable++ = rsvd_ISR;
    }

    EDIS;  // Disable writes to protected registers

    // Enable the PIE Vector Table
    PieCtrlRegs.PIECTRL.bit.ENPIE = 1;
}
```

PieVectTable	struct PIE_VEC...	{PIE1_RESERVED=0x...	0x00000D00@Data
PIE1_RESERVED	void (*)()	0x965B2A90	0x00000D00@Data
PIE2_RESERVED	void (*)()	0x00000000	0x00000D02@Data
PIE3_RESERVED	void (*)()	0x01400000	0x00000D04@Data
PIE4_RESERVED	void (*)()	0x000082E1	0x00000D06@Data

1.3 更改保留函数

1、保留中断函数所有代码需要在 RAM 执行；

2、关闭中断；

3、重新写封波函数，确保（EPWM2A/2B,EPWM3A/3B,EPWM4A/4B 内连引脚全部置低）；

4、重新使能看门狗，确保进入保留中断函数看门狗可工作；

5、重新配置 SCL 为输入，SDA 为 TZ；

6、保留中断函数结尾增加 while(1)，确保函数停在保留中断中运行。

```
398#pragma CODE_SECTION(rsvd_ISR, "ramfuncs");
399interrupt void rsvd_ISR(void)
400 {
401    DINT;
402    DisableDrive();                          //停机封锁输出
403    EALLOW;
404    /*scl 配置为输入*/
405    GpioCtrlRegs.GPAMUX2.bit.GPIO29 = 0;
406    GpioCtrlRegs.GPADIR.bit.GPIO29  = 0;
407    /*sda 配置tz*/
408    GpioCtrlRegs.GPAMUX2.bit.GPIO28 = 3;
409    EDIS;
410    EALLOW;
411    SysCtrlRegs.WDCR= 0x0028;
412    EDIS;
413    while(1);  // 确保异常发生程序停留在RAM
414}
415
```

1.4 确认软件配置 Flash 擦除时间及更新 FlashAPI

根据 CPU 主频配置 CPU_RATE，API 会根据该值自动匹配 Flash 操作时间。

下图为 100M 配置示例：

```
EALLOW;  // Enable writes to protected registers
Flash_CPUScaleFactor = SCALE_FACTOR;  // Set CPU scaling factor for Flash timing
Flash_CallbackPtr = 0;                // Initialize callback pointer
EDIS;   // Disable writes to protected registers
```

```
66 ------------------------------------------------------------------------------*/
67 #define CPU_RATE    10.000L   // for a 100MHz CPU clock speed (SYSCLKOUT)
68 //#define CPU_RATE   16.667L   // for a 60MHz CPU clock speed (SYSCLKOUT)
69 //#define CPU_RATE   20.000L   // for a 50MHz CPU clock speed  (SYSCLKOUT)
70 //#define CPU_RATE   25.000L   // for a 40MHz CPU clock speed  (SYSCLKOUT)
71 //#define CPU_RATE   33.333L   // for a 30MHz CPU clock speed  (SYSCLKOUT)
72 //#define CPU_RATE   41.667L   // for a 24MHz CPU clock speed  (SYSCLKOUT)
73 //#define CPU_RATE   50.000L   // for a 20MHz CPU clock speed  (SYSCLKOUT)
74 //#define CPU_RATE   66.667L   // for a 15MHz CPU clock speed  (SYSCLKOUT)
75 //#define CPU_RATE  100.000L   // for a 10MHz CPU clock speed  (SYSCLKOUT)
76
77 //---------------------------------------------------------------------------
78
79
80 //---------------------------------------------------------------------------
81 // **** DO NOT modify the code below this line ****
82 //---------------------------------------------------------------------------
83 #define SCALE_FACTOR  1048576.0L*( (200L/CPU_RATE) )  // IQ20
```

更新 API 程序，版本如下：



ADP32F036_027_
Flash_API_AUTO(

2 APP 程序

2.1 更改 Flash 等待时间

100M 主频，Flash 等待时间 3，3，5 改成 4，4，5。

```
4 void InitFlash(void)
5 {
6    EALLOW;
7
8    //Enable Flash Pipeline mode to improve performance of code executed from Flash.
9    //FlashRegs.FOPT.bit.ENPIPE = 1;
0    FlashRegs.FOPT.all = 0x01;
1    //Set the Random Waitstate for the Flash
2    FlashRegs.FBANKWAIT.bit.RANDWAIT = READ_RAND_FLASH_WAITE;
3    //Set the Paged Waitstate for the Flash
4    FlashRegs.FBANKWAIT.bit.PAGEWAIT = READ_PAGE_FLASH_WAITE;
5    //Set the Waitstate for the OTP
6    FlashRegs.FOTPWAIT.bit.OTPWAIT = READ_OTP_WAITE;
7
8    //Set number of cycles to transition from sleep to standby
9    //FlashRegs.FSTDBYWAIT.bit.STDBYWAIT = 0x01FF;
0    //Set number of cycles to transition from standby to active
1    //FlashRegs.FACTIVEWAIT.bit.ACTIVEWAIT = 0x01FF;
2    EDIS;
3
4    asm(" RPT #7 || NOP");
5 }
```

```
/*************************************************************
    定义Flash 等待时间：035B/036 相同。
*************************************************************/
#define   READ_RAND_FLASH_WAITE       4       //Flash中运行的等待时间
#define   READ_PAGE_FLASH_WAITE       4       //
#define   READ_OTP_WAITE              5       //OTP中读数据的等待时间
```

2.2 更改中断向量表

地址 0xD00~0xD05 不属于中断向量，需跳过。

添加 pPieTable += 3，将 128 改成 125，需要确定 PieVectTable 的起始地址为 0xD00。

HardwareSetup.c

```
427
428 //---------------------------------------------------------------------------
429 //初始化的时候首先把所有的中断服务程序 都初始化为默认服务程序
430 //---------------------------------------------------------------------------
431 void InitPieVectTable(void)
432 {
433     int16   i;
434     PINT *pPieTable = (void *) &PieVectTable;
435     pPieTable += 3;
436     EALLOW;
437     for(i=0; i < 125; i++)
438     {
439         *pPieTable++ = rsvd_ISR;
440     }
441     EDIS;
442
443     // Enable the PIE Vector Table
444     PieCtrlRegs.PIECTRL.bit.ENPIE = 1;
445 }
446
```

PieVectTable	struct PIE_VEC...	{PIE1_RESERVED=0x...	0x00000D00@Data
PIE1_RESERVED	void (*)()	0x965B2A90	0x00000D00@Data
PIE2_RESERVED	void (*)()	0x00000000	0x00000D02@Data
PIE3_RESERVED	void (*)()	0x01400000	0x00000D04@Data
PIE4_RESERVED	void (*)()	0x000082E1	0x00000D06@Data

2.3 更改保留函数

1、保留中断函数所有代码需要在 RAM 执行；

2、关闭中断；

3、重新写封波函数，确保（EPWM2A/2B,EPWM3A/3B,EPWM4A/4B 内连引脚全部置低）；

4、重新使能看门狗，确保进入保留中断函数看门狗可工作；

5、重新配置 SCL 为输入，SDA 为 TZ；

6、保留中断函数结尾增加 while(1)，确保函数停在保留中断中运行。

```
#pragma CODE_SECTION(rsvd_ISR, "ramfuncs");
interrupt void rsvd_ISR(void)
 {
    DINT;
    DisableDrive();                          //停机封锁输出
    EALLOW;
    /*scl 配置为输入*/
    GpioCtrlRegs.GPAMUX2.bit.GPIO29 = 0;
    GpioCtrlRegs.GPADIR.bit.GPIO29  = 0;
    /*sda 配置tz*/
    GpioCtrlRegs.GPAMUX2.bit.GPIO28 = 3;
    EDIS;
    EALLOW;
    SysCtrlRegs.WDCR= 0x0028;
    EDIS;
    while(1);  // 确保异常发生程序停留在RAM
}

```

2.4 休眠函数例程建议

1、休眠函数里所有代码需要保证在 RAM 运行；

2、在 LIN_EN 拉低前一定要确保 DSP 封波；

（ADM32F036A2/3 内连引脚 EPWM2A/2B,EPWM3A/3B,EPWM4A/4B，需全部置低）；

3、休眠时序要满足 LIN_EN 拉低 3us 以内，拉低 LIN_TXD 且低电平维持到 DSP 完全下电；

4、让芯片休眠后完全掉电（VDDIO 完全掉电到 0 V）后再进行唤醒操作（在此之前 LIN 总线不能再发数据），避免休眠异常唤醒状态。如果 LIN 总线上由于受到干扰导致休眠失败，可以增加软件逻辑功能；

5、休眠/VM 下电前可通过配置 SCL 为输入、SDA 为 TZ 模式、降频等措施，降低 VDDIO 下电速率，确保休眠/VM 下电过程 VDD 处于设计规范（1.08V）以上。

```
#pragma CODE_SECTION(mcu_sleep, "ramfuncs");
void mcu_sleep(void)
{
    DINT;
    DisableDrive();//停机封锁输出
    EALLOW;
    /*scl 配置为输入*/
    GpioCtrlRegs.GPAMUX2.bit.GPIO29 = 0;
    GpioCtrlRegs.GPADIR.bit.GPIO29  = 0;

    /*sda 配置tz*/
    GpioCtrlRegs.GPAMUX2.bit.GPIO28 = 3;        //TZmode
    EDIS;

    EALLOW;
    SysCtrlRegs.PCLKCR0.all = 0;    // 关闭外设时钟
    SysCtrlRegs.PCLKCR1.all = 0;    // 关闭外设时钟
    SysCtrlRegs.PCLKCR3.all = 0;    // 关闭外设时钟
    EDIS;

    EALLOW;
    SysCtrlRegs.PLLCR.bit.DIV = 6;
    SysCtrlRegs.PLLSTS.bit.DIVSEL = 3;//降频至60M;
    EDIS;
    ADP32F03x_usDelay(20000L);

    GpioDataRegs.GPACLEAR.bit.GPIO1 = 1;//  LINEN 对应的IO 拉低
    asm(" RPT #50 || NOP ");
    EALLOW;
    GpioCtrlRegs.GPAMUX1.bit.GPIO14 = 0;
    GpioCtrlRegs.GPADIR.bit.GPIO14 = 1;
    GpioDataRegs.GPACLEAR.bit.GPIO14 = 1; // LINTX 拉低
    EDIS;
    ADP32F03x_usDelay(20000L);
    EALLOW;
    SysCtrlRegs.WDCR=0x0000;//看门狗复位
    EDIS;
    while(1);
}
```

2.5 确认软件配置 Flash 擦除时间及更新 FlashAPI

根据 CPU 主频配置 CPU_RATE，API 会根据该值自动匹配 Flash 操作时间。

下图为 100M 配置示例：

```
EALLOW;  // Enable writes to protected registers
Flash_CPUScaleFactor = SCALE_FACTOR;  // Set CPU scaling factor for Flash timing
Flash_CallbackPtr = 0;                // Initialize callback pointer
EDIS;   // Disable writes to protected registers
```

```
66 ------------------------------------------------------------------------------*/
67 #define CPU_RATE    10.000L   // for a 100MHz CPU clock speed (SYSCLKOUT)
68 //#define CPU_RATE   16.667L   // for a 60MHz CPU clock speed (SYSCLKOUT)
69 //#define CPU_RATE   20.000L   // for a 50MHz CPU clock speed  (SYSCLKOUT)
70 //#define CPU_RATE   25.000L   // for a 40MHz CPU clock speed  (SYSCLKOUT)
71 //#define CPU_RATE   33.333L   // for a 30MHz CPU clock speed  (SYSCLKOUT)
72 //#define CPU_RATE   41.667L   // for a 24MHz CPU clock speed  (SYSCLKOUT)
73 //#define CPU_RATE   50.000L   // for a 20MHz CPU clock speed  (SYSCLKOUT)
74 //#define CPU_RATE   66.667L   // for a 15MHz CPU clock speed  (SYSCLKOUT)
75 //#define CPU_RATE  100.000L   // for a 10MHz CPU clock speed  (SYSCLKOUT)
76
77 //---------------------------------------------------------------------------
78
79
80 //---------------------------------------------------------------------------
81 // **** DO NOT modify the code below this line ****
82 //---------------------------------------------------------------------------
83 #define SCALE_FACTOR  1048576.0L*( (200L/CPU_RATE) )  // IQ20
```

更新 API 程序，版本如下：



ADP32F036_027_
Flash_API_AUTOC

2.6 休眠/VM 下电到 5.3V 前（可通过 ADC 采样母线判断），确保 SCL 为输入模式，SDA 为 TZ 模式

Step1 GPIO 初始化函数中将 SCL,SDA 配置为 IO 输入模式，

```
32
33 void InitI2CIO(void)
34 {
35     EALLOW;
36     GpioCtrlRegs.GPAPUD.bit.GPIO28 = 0;
37     GpioCtrlRegs.GPAPUD.bit.GPIO29 = 0;
38     GpioCtrlRegs.GPAMUX2.bit.GPIO28 = 0;
39     GpioCtrlRegs.GPAMUX2.bit.GPIO29 = 0;
40     GpioCtrlRegs.GPADIR.bit.GPIO28  = 0;
41     GpioCtrlRegs.GPADIR.bit.GPIO29  = 0;
42     EDIS;
43 }
```

Step2.1 增加 SCL_IN,SCL_OUT 宏定义

```
41
42 #define SCL_IN                  {EALLOW; GpioCtrlRegs.GPADIR.bit.GPIO29 = 0; EDIS;}
43 #define SCL_OUT                 {EALLOW; GpioCtrlRegs.GPADIR.bit.GPIO29 = 1; EDIS;}
44
```

Step2.2 预驱读写前配置 SCL，SDA 为输出，用于 IIC 时钟模拟，读写完成

后配置 SCL 为输入

```
44
45 static void PreDriverWriteData(Uint16 Addr, Uint16 Data)
46 {
47     Uint16 i;
48     Uint16 SendData;
49     SCL_OUT
50     //start
51     SDA_OUT;
52     SDA_H;
53     SCL_H;
54     ADP32F03x_usDelay(H_LEVEL_DELAY);
```

写函数起始信号配置 SCL 输出

```
139
140     //STOP
141     SDA_OUT;
142     SDA_L;
143     ADP32F03x_usDelay(L_LEVEL_DELAY);
144     SCL_H;
145     ADP32F03x_usDelay(STOP_DELAY);
146     SDA_H;
147
148     SCL_IN
149 }
```

写函数停止信号配置 SCL 输入

```
151 static Uint16 PreDriverReadData(Uint16 Addr)
152 {
153     Uint16 i;
154     Uint16 SendData;
155     Uint16 ReadData;
156     SCL_OUT
157     //start
158     SDA_OUT;
159     SDA_H;
160     SCL_H;
```

读函数起始信号配置 SCL 输出

```
244     //STOP
245     ADP32F03x_usDelay(L_LEVEL_DELAY);
246     SCL_H;
247     ADP32F03x_usDelay(STOP_DELAY);
248     SDA_H;
249     SCL_IN
250     return(ReadData);
251 }
252
```

读函数停止信号配置 SCL 输入

联系方式

公司网址：www.advancechip.com

联系邮箱：sales@advancechip.com

销售联系电话：0731-88731027（长沙）

公司总部地址：长沙市湘江新区东方红街道北斗产业园•黄金园 A5 栋

南京销售中心：南京市秦淮区卡子门大街 19 号紫云智慧广场 6 号楼 15 层